Support Vector Machine From Scratch

MGSC 696

April 30th, 2026

David Fogel

Sebastian Arguedas Soley

Lucas Penney

Aziz Ahmed

Fares Jony

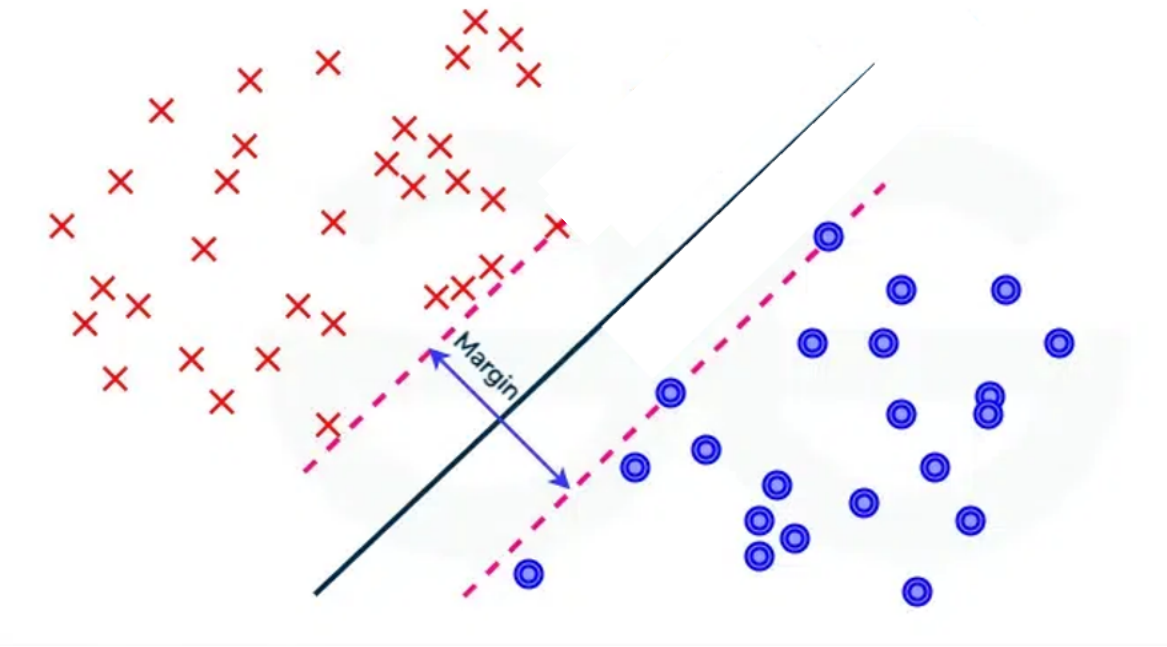
1. Introduction and Method Description

Support Vector Machines (SVMs) are supervised learning models designed for binary classification. The goal is straightforward: given labelled training data, find the best possible line (or plane, in higher dimensions) that separates the two classes. What makes SVMs different from other classifiers is how "best" is defined, the SVM specifically finds the separation that leaves the largest possible gap between the two classes. This gap is called the margin**,** and maximising it leads to better performance on new, unseen data.

1.1 The Margin

The separating boundary (called a hyperplane) is defined by two parameters: a weight vector w, which controls the orientation of the boundary, and a bias term b, which shifts it. Any point x can be evaluated by plugging it into $w \cdot x + b$. a positive result places it on one side, a negative result on the other.

The SVM places two parallel boundary lines on either side of the main separator, one touching each class. The distance between these two lines is the margin, which equals $2 / \|w\|$. A shorter w vector means a wider gap, since margin = $2/\|w\|$, making the denominator smaller makes the fraction bigger. Therefore, maximising this margin is the same as minimising $\|w\|^2$, which is the core of the SVM objective.



1.2 Hard Margin vs. Soft Margin

In the simplest case (hard margin), every training point must be correctly classified and outside the margin:

Minimize

$(1/2)\|w\|^2$

Subject to

$y_i(w \cdot x_i + b) \geq 1 \quad \text{for all } i$

where $y_i \in \{-1, +1\}$ is the label of point i. This works only when the data is perfectly separable, which is rarely true in practice.

The soft margin relaxes this by allowing some points to be inside the margin or even on the wrong side. Each point gets a slack variable $\xi_i \geq 0$ that measures how much it violates the constraint:

Minimize

$(1/2)\|w\|^2 + C \cdot \Sigma\, \xi_i$

Subject to

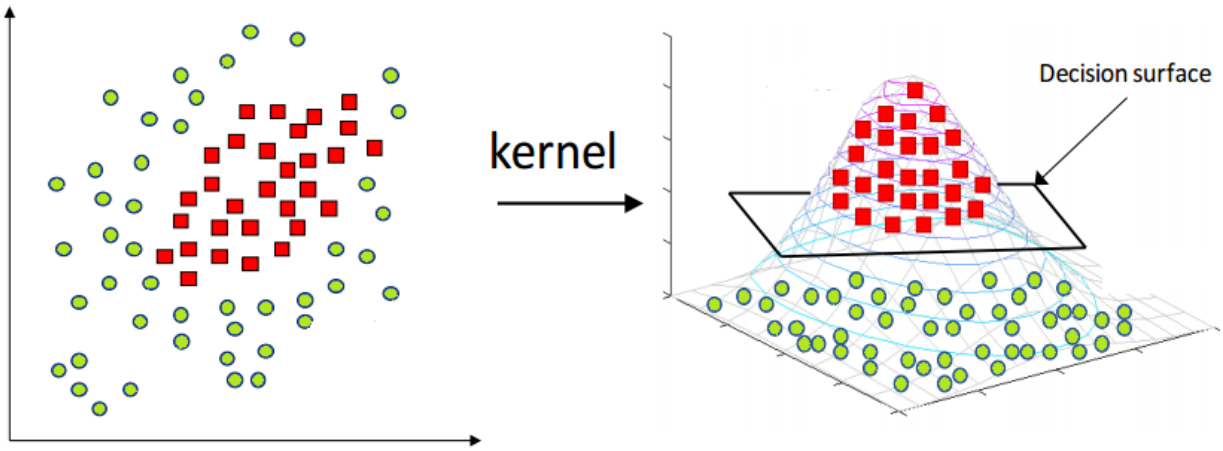
$y_i(w \cdot x_i + b) \geq 1 - \xi_i$

$\xi_i \geq 0$

The parameter C controls the trade-off: a large C heavily penalises any violation, pushing the model toward a tight fit of the training data; a small C allows more violations in exchange for a wider, more robust margin.

1.3 The Kernel Trick

The formulations above find linear boundaries. For data that is not linearly separable, the kernel trick allows the SVM to find non-linear boundaries without explicitly transforming the data. A kernel function $K(x_i, x_j)$ computes the similarity between two points in a higher-dimensional space, without ever computing the mapping itself.



Three kernels were implemented:

- Linear: $K(x_i, x_j) = x_i \cdot x_j$ equivalent to no transformation; the boundary is a straight line/plane in the original space.
- Polynomial: $K(x_i, x_j) = (x_i \cdot x_j + c)^d$ implicitly maps to a space containing all combinations of features up to degree d, enabling curved boundaries.
- RBF (Gaussian): $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ measures closeness between points; nearby points are considered similar, far-apart points are not. The parameter γ controls how quickly similarity drops off with distance. The RBF kernel does not transform the data, it simply measures the distance between two points and converts it to a similarity score between 0 and 1.

2. Optimization Problem and Solution

2.1 Reformulating as a Dual Problem

To solve the soft-margin SVM, we rewrite it in a form that is easier for a computer to handle. We introduce a new variable α_i for each training point, these will tell us which points become support vectors. We then combine the objective and all the constraints into one single expression and find the values of α_i that minimise it. This gives us three useful results:

(i) $w = \Sigma\, \alpha_i\, y_i\, x_i$
(ii) $\Sigma\, \alpha_i\, y_i = 0$
(iii) $\alpha_i \leq C$

Result (i) is important: the weight vector is simply a weighted sum of the training points, where most weights α_i will be zero. Only points that sit on or inside the margin, called support vectors, have $\alpha_i > 0$. This is where the name comes from.

Substituting these results back eliminates w, b, and ξ entirely, giving the dual problem:

Maximise

$\Sigma\alpha_i - (1/2) \cdot \Sigma_i\Sigma_j \alpha_i \alpha_j y_i y_j K(x_i, x_j)$

Subject to

$0 \leq \alpha_i \leq C$ for all i,

$\Sigma \alpha_i y_i = 0$

The data only appears as dot products $K(x_i, x_j)$, which is why any kernel can be dropped in without changing the structure of the problem.

2.2 Problem Classification

The dual problem is a convex quadratic program (QP):

- The objective is quadratic in the variables α_i (due to the $\alpha_i\alpha_j$ terms), not linear
- All constraints are linear (box constraints and one equality)
- The problem is convex because valid kernel functions are mathematically guaranteed to produce a bowl-shaped objective, meaning there is only one correct answer and the solver cannot get stuck.

2.3 Solving the QP with cvxopt

The dual QP was solved using cvxopt, a general-purpose convex optimisation library. It uses an interior-point method (an algorithm that navigates toward the optimal solution from inside the feasible region, rather than along its edges*)* to find the optimal α_i values. cvxopt has no knowledge of SVMs, All SVM-specific logic (building the kernel matrix, setting up the constraints, extracting support vectors, computing predictions) was implemented from scratch.

After solving, support vectors are identified as points where $\alpha_i > 10^{-5}$. The bias b is computed from support vectors that lie exactly on the margin ($0 < \alpha_i < C$), averaged for numerical stability:

$b = y_s - \Sigma \alpha_i y_i K(x_i, x_s)$

To classify a new point x, the prediction is:

$\hat{y} = \text{sign}(\Sigma \alpha_i y_i K(x_i, x) + b)$ summed over support vectors only

3. Binary Classification Only

This implementation is restricted to binary classification, problems where each data point belongs to one of exactly two classes. The SVM formulation described above is inherently binary: labels are encoded as +1 and −1, and the objective finds a single separating hyperplane between them.

3.1 Why Multi-Class Was Not Implemented

Extending SVMs to more than two classes requires additional strategies, since the core algorithm only produces a two-class decision. The two standard approaches are:

- One-vs-Rest (OvR): Train one binary SVM per class, where that class is treated as positive and all others as negative. To predict, run all classifiers and pick the one with the highest confidence score.
- One-vs-One (OvO): Train one binary SVM for every pair of classes. For k classes, this produces $k(k-1)/2$ classifiers. To predict, each classifier votes and the class with the most votes wins.

Both strategies add significant complexity. OvR requires careful handling of class imbalance (since one class is always in the minority), and OvO multiplies the number of models by a quadratic factor. More importantly, either extension would shift focus away from the core SVM mechanics; the dual formulation, kernel trick, and support vector geometry.

The Breast Cancer Wisconsin dataset is naturally binary (malignant vs. benign), making it an ideal fit for the binary SVM. No multi-class extension was needed for meaningful, real-world experiments.

4. Experimental Analysis

4.1 Dataset

The Breast Cancer Wisconsin (Diagnostic) dataset was used. It contains 569 samples with 30 numerical features computed from images of cell nuclei in biopsy samples. The features describe

properties such as radius, texture, perimeter, smoothness, and concavity at three scales (mean, standard error, and worst value). The target label is malignant (212 samples, $y = -1$) or benign (357 samples, $y = +1$).

Preprocessing: Labels were remapped from $\{0, 1\}$ to $\{-1, +1\}$. All features were standardised to zero mean and unit variance using training set statistics, this is essential because SVMs are sensitive to feature scale. A 75/25 stratified train-test split produced 426 training and 143 test samples.

4.2 Comparison with sklearn

Three kernel variants were evaluated (linear, polynomial degree 3, and RBF with $\gamma = 0.03$), all with $C = 1.0$. The from-scratch implementation was compared against sklearn.svm.SVC, which uses the libsvm SMO algorithm internally.

Kernel	Implementation	Accuracy	# SVs	Train time (s)
Linear	Scratch	0.9860	34	0.033
Linear	sklearn	0.9860	34	0.002
Polynomial	Scratch	0.9650	63	0.019
Polynomial	sklearn	0.9720	65	0.001
RBF	Scratch	0.9790	95	0.011
RBF	sklearn	0.9790	92	0.001

Test accuracy, support vector count, and training time. C = 1.0 for all models.

Accuracy was near-identical across implementations for all kernels, confirming both solvers reach the same optimal hyperplane. The main difference was speed: sklearn's SMO algorithm is significantly faster because it solves the QP analytically using two-variable subproblems, while cvxopt's interior-point method is a general solver and does not exploit the structure of the SVM problem.

As a further validation, the predictions of the scratch and sklearn implementations were compared point-by-point on the 143 test patients. The linear and RBF kernels achieved perfect prediction agreement (143/143), confirming both solvers converged to the exact same decision boundary. The polynomial kernel showed agreement on 142/143 points, with the 1 disagreement consistent with the small accuracy gap between implementations, a result of numerical differences between cvxopt's interior-point method and sklearn's SMO algorithm.

4.3 Performance Metrics

For a medical dataset, recall on the malignant class is the most important metric, missing a malignant tumour (false negative) is a more serious error than a false alarm (false positive).

Class	Precision	Recall	F1
Malignant	0.98	0.98	0.98
Benign	0.99	0.99	0.99

Classification report for best from scratch model (Linear kernel).

Out of 143 test patients, only 2 were misclassified, 1 malignant patient predicted as benign (the more dangerous error), and 1 benign patient predicted as malignant.

4.4 Effect of C on Support Vectors

The model was trained across C ∈ {0.01, 0.1, 0.5, 1, 5, 10, 100} using the RBF kernel. At low C, many points become support vectors because the wide margin tolerates violations from a large portion of the training set. As C increases, the margin tightens and fewer points are needed to define it. C=5 produced the highest test accuracy at 0.9860, though C=1 offers a comparable 0.9790 with a slightly wider margin, making it a more conservative and generalizable choice. Beyond C=10, accuracy drops noticeably as the model begins to overfit.

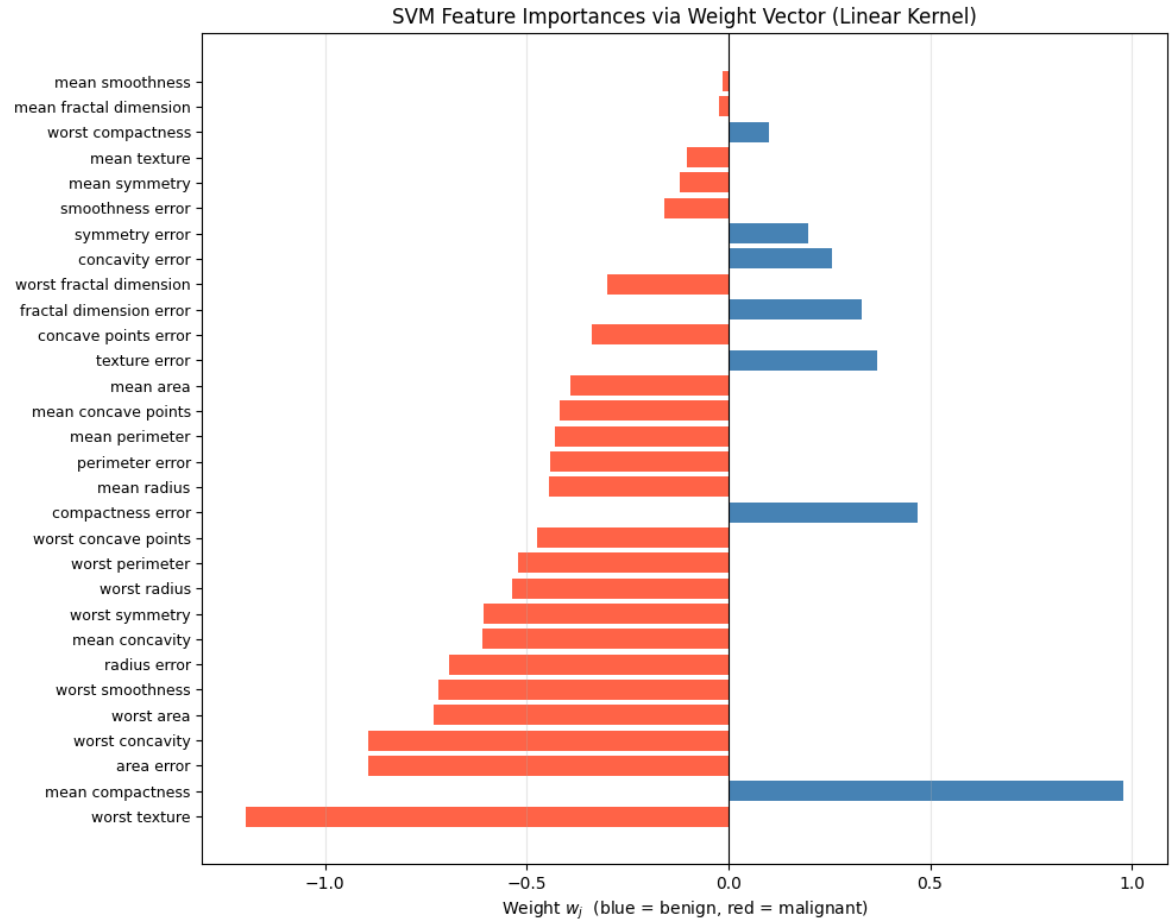
C	# SVs	Test Accuracy
0.01	322	0.6294
0.1	189	0.9371
0.5	116	0.9720
1	95	0.9790
5	77	0.9860
10	74	0.9790
100	67	0.9441

4.5 Feature Importances (Linear Kernel)

The most influential features were worst texture, area error, and worst concavity. Many of the top features push toward the malignant class (negative weights), with mean compactness being the strongest benign indicator. This is consistent with medical knowledge that irregular, larger cell nuclei are associated with malignancy.

Also worth noting: 9 out of the top 10 features lean malignant, which tells you the model is largely learning what malignant looks like rather than what benign looks like.

The weight vector was also compared against sklearn's LinearSVC by computing the cosine similarity between the two normalised vectors. A result near 1.0 confirms both optimisers converged to the same separating hyperplane despite using completely different algorithms.



5. Implementation Challenges

Bias computation stability: The bias b should be recoverable from any support vector sitting exactly on the margin ($0 < \alpha_i < C$). In practice, floating-point errors from the QP solver mean few α_i values land precisely in this range. This was resolved by averaging b over all margin support vectors, with a fallback to all support vectors when none qualified.

Identifying support vectors: Solved α_i values are never exactly zero due to numerical precision. A threshold of 10^{-5} was used to distinguish true support vectors from near-zero noise. A threshold that is too high incorrectly discards valid support vectors; too low and noise pollutes the bias estimate.

Kernel matrix conditioning: The polynomial kernel at high degree produces very large Gram matrix values, which cause numerical instability in the QP solver. This was addressed by fixing degree = 3 and coef0 = 1, which kept matrix values well-conditioned.

All three challenges were successfully resolved, and the final implementation produces accuracy results matching sklearn to within numerical tolerance across all three kernels.

6. Conclusion

This project implemented a binary soft-margin SVM from scratch using the dual QP formulation, with support for linear, polynomial, and RBF kernels. The implementation was validated against sklearn's SVC and produced near-identical accuracy across all three kernels, confirming the correctness of the approach.

The Breast Cancer Wisconsin dataset proved to be a strong fit for the binary SVM, the two classes are well-separated in the standardised feature space, and the model achieved 99% test accuracy with only 2 misclassifications out of 143 test patients. The one missed malignant case highlights the real-world cost of false negatives in medical classification and motivates the use of recall as the primary evaluation metric over raw accuracy.

The key insight from this project is that the SVM's power comes not from the classifier itself, but from the combination of the maximum-margin objective, the dual formulation that reduces the problem to finding a small set of support vectors, and the kernel trick that allows non-linear boundaries without any additional implementation complexity. The cvxopt QP solver handled the optimisation correctly and produced results consistent with sklearn, though the speed gap confirms why specialised solvers like SMO are used in production systems.

References

M. Andersen, J. Dahl, and L. Vandenberghe, "CVXOPT: A Python package for convex optimization," Version 1.3, 2023. Available at: https://cvxopt.org/

W. H. Wolberg, W. N. Street, and O. L. Mangasarian, "Breast Cancer Wisconsin (Diagnostic) Data Set," UCI Machine Learning Repository, 1995. Available at: https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(diagnostic)

Scikit-learn developers, "Support Vector Machines," scikit-learn documentation. Available at: https://scikit-learn.org/stable/modules/svm.html

Scikit-learn developers, "sklearn.svm.SVC documentation." Available at: https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

Stanford University CS229, "Support Vector Machines Lecture Notes," Andrew Ng. Available at: https://cs229.stanford.edu/notes2022fall/main_notes.pdf